



# Fraud Risk Intelligence Platform

SQL-first fraud detection · **XGBoost** · expected-loss alert queue

Nihira Sharma · PostgreSQL + Python · Sparkov dataset (1.3M transactions)

## THE PROBLEM

# Catching fraud is easy. Catching it precisely is not.

Fraud is rare and expensive. A rule engine can flag it, but at terrible precision: **it raises roughly five false alarms for every real fraud**, which buries a finite analyst team. The real lever is precision: how much fraud you recover for each alert you can afford to review.

### FRAUD RATE

**0.58%**

7,506 of 1.3M transactions

### RULE-ENGINE PRECISION

**17%**

at 43.9% recall

### FALSE ALARMS PER FRAUD

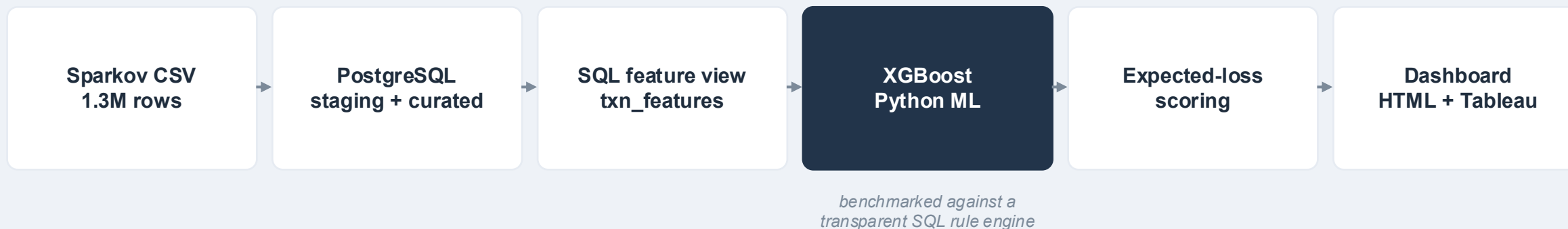
**~5 : 1**

the cost of low precision

*Goal: lift precision far above the rule baseline at the same recall, then rank alerts so a fixed review budget recovers the most fraud.*

## ARCHITECTURE

# A SQL-first pipeline



**All feature engineering lives in PostgreSQL:** trailing-window velocity counts, per-card amount z-scores, and haversine distances are computed as a single materialized view. Python handles only model training and scoring. This keeps the heavy data work database-native and the pipeline reproducible.

# Which features actually separate fraud?

## Amount deviation

### DOMINANT SIGNAL

Per-card amount z-score: fraud sits ~3.2 standard deviations above a card's normal spend vs ~0 for legit. The single strongest feature.

## Geo features

### NO SIGNAL

Home-to-merchant distance is identical across classes (76.1 vs 76.3 km). Sparkov scatters coordinates randomly, so distance is dead. Documented as a negative result; impossible-travel speed dropped.

## Velocity & time

### MILD LIFT

Trailing-24h count slightly higher for fraud (5.23 vs 4.88), and night-time fraud rate (1.03%) is ~2.2x the daytime rate. Kept with modest weight.

*Features were engineered from domain reasoning, then empirically tested. The model matrix reflects what earned its place, not a kitchen sink.*

RESULTS

The model beats the rule baseline decisively

| Model                           | PR-AUC | Precision @ 43.9% recall |
|---------------------------------|--------|--------------------------|
| XGBoost (production)            | 0.883  | 99.2%                    |
| LightGBM                        | 0.870  | 98.5%                    |
| Random Forest                   | 0.846  | 98.3%                    |
| Logistic Regression             | 0.402  | 53.5%                    |
| Isolation Forest (unsupervised) | 0.326  | 45.8%                    |
| Rule engine (SQL baseline)      | —      | 17.0%                    |

17% → 99.2%

precision at the same 43.9% recall as the rule engine

XGBoost · PR-AUC 0.883 · 198,982-txn time-based hold-out

Scored on a time-based split (train before Apr 2020, test Apr–Jun 2020). Isolation Forest is a deliberate unsupervised contrast: it underperforms when labels exist.

## THE CORE IDEA

# From classifier to capacity-aware review queue

Rank every alert by expected loss = **P(fraud) × transaction amount**. Analysts work down the queue against a fixed review budget, so the dashboard reads off exactly how much fraud and how many dollars a given staffing level recovers.

ALERTS REVIEWED

**1,000**

of 2,730 gated

OF ALL FRAUD CAUGHT

**75%**

872 of 1,163

ALERT PRECISION

**87%**

vs 17% rule baseline

FRAUD \$ RECOVERED

**\$529K**

85% of loss exposure

*Reviewing the top 1,000 of 2,730 flagged alerts recovers three-quarters of all fraud and 85% of the fraud-dollar exposure.*

## DELIVERY

# A live operational dashboard

ALERTS REVIEWED

1,000

FRAUD CAUGHT

872

PRECISION

87%

FRAUD \$ RECOVERED

\$529,621

### Single-screen risk console

KPI band, an alert-budget curve, an expected-loss-ranked alert queue, and analytics tiles. Built with Plotly.js and vanilla JS, hosted free on GitHub Pages so the link is always on.

### LIVE LINKS

**Dashboard** [nihira11.github.io/.../dashboard](https://nihira11.github.io/.../dashboard)

**Tableau** [public.tableau.com/.../Fraud\\_Risk](https://public.tableau.com/.../Fraud_Risk)

**Code** [github.com/nihira11/fraud-risk-...](https://github.com/nihira11/fraud-risk-...)

# What this project demonstrates

- **SQL-first engineering**  
Rolling windows, per-card z-scores and haversine distances built as a PostgreSQL materialized view.
- **Modeling with judgement**  
A five-model bake-off scored on a time-based split against a transparent rule baseline, not a single black box.
- **Business framing**  
Expected-loss ranking turns probabilities into a capacity-aware queue measured in fraud and dollars recovered.
- **End-to-end delivery**  
From raw CSV to a live, always-on dashboard and a Tableau workbook.

*Honest caveat: Sparkov is synthetic, so the precision figures are optimistic relative to production fraud.*